

PLANIMO - TRAVEL PACKAGE MARKETPLACE

Submitted by

RAGHAVAN S
MYMOON BEEVI B

2116230701520
2116230701201

in partial fulfilment of the award of the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING



RAJALAKSHMI
ENGINEERING COLLEGE
An AUTONOMOUS Institution
Affiliated to ANNA UNIVERSITY, Chennai



RAJALAKSHMI ENGINEERING COLLEGE

ANNA UNIVERSITY, CHENNAI

MAY 2026

BONAFIDE CERTIFICATE

Certified that this Project titled “PLANYMO - TRAVEL PACKAGE MARKETPLACE ” is the Bonafide work of “RAGHAVAN S (2116230701520) and MYMOON BEEVI B (2116230701)” who carried out the work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE

Mrs. Deepa ,

M.E(CSE),

Computer Science & Engineering,

Rajalakshmi Engineering College

Thandalam, Chennai -602105

Submitted for the End semester practical examination to be held on

INTERNAL EXAMINER

EXTERNAL EXAMINER

ABSTRACT

Planymo is a full-stack travel marketplace platform that connects travelers with vendors through a unified and scalable system. It addresses common issues in existing platforms such as fragmented services, lack of trust, and poor performance during high traffic. The application is built using Flutter for cross-platform mobile development and Django REST-based microservices for the backend, ensuring modularity and scalability. An event-driven architecture using Apache Kafka enables efficient asynchronous communication, while the Transactional Outbox Pattern ensures data consistency. Redis-based distributed locking prevents double-booking during concurrent transactions. Security is implemented using OAuth2 and JWT with PKCE for safe authentication. The platform also includes a booking-verified review system to enhance trust. Overall, Planymo demonstrates a reliable, scalable, and secure solution for modern travel booking systems, with scope for future enhancements such as AI-driven recommendations.

ACKNOWLEDGEMENT

Initially, we thank the Almighty for being with us through every walk of our life and showering his blessings through the endeavour to put forth this report. Our sincere thanks to our chairman **Mr. S. MEGANATHAN, B.E, F.I.E.**, our respected Chairperson **Dr. (Mrs.) THANGAM MEGANATHAN, Ph.D.**, our Vice Chairman **Mr. ABHAY SHANKAR MEGANATHAN, B.E, M.S.**, for providing us with the requisite infrastructure and sincere endeavouring in educating us in their premier institution.

Our sincere thanks to **Dr. S.N. MURUGESAN, M.E., Ph.D.**, our beloved principal for his kind support and facilities provided to complete our work in time. We express our sincere thanks to **Dr. E. M. Malathy, M.E., Ph.D.**, Professor and Head of the Department of Computer Science and Engineering and our mentor **MRS. DEEPA B, M.E(CSE).**, Department of Computer Science and Engineering for his useful tips during our review to build our project.

TABLE OF CONTENTS		
CHAPTER	TITLE	PAGE NO
	ABSTRACT	iii
	ACKNOWLEDGEMENT	iv
1.	INTRODUCTION	06
	1.1 GENERAL	06
	1.2 SCOPE OF THE WORK	06
	1.3 AIM & OBJECTIVES OF THE PROJECT	07
2.	LITERATURE SURVEY	08
3.	SYSTEM DESIGN	09
	3.1 ARCHITECTURE DIAGRAM	09
	3.2 USE CASE DIAGRAM	10
	3.3 HARDWARE SPECIFICATIONS	11
	3.4 SOFTWARE SPECIFICATIONS	12
4.	PROPOSED SYSTEM	13
5.	MODULE DESCRIPTION	14
6.	OUTPUT	16
7.	CONCLUSION AND FUTURE ENHANCEMENTS	17
8.	REFERENCES	18

CHAPTER 1

INTRODUCTION

1.1 GENERAL

Planymo is a full-stack travel marketplace application designed to connect travelers with vendors through a unified and scalable digital platform. Built using Flutter for cross-platform mobile development and a microservices-based backend with Django REST Framework, the system delivers a seamless and high-performance booking experience. It addresses key challenges in existing travel platforms such as fragmented services, lack of trust, and performance issues during high user traffic. The application includes core modules such as authentication, package discovery, booking and reservation, payment processing, vendor management, reviews, and support services, each operating as an independent microservice to ensure scalability and fault isolation.

The system adopts an event-driven architecture using Apache Kafka for asynchronous communication, improving responsiveness and reliability. To maintain data consistency and prevent double-booking, it implements the Transactional Outbox Pattern along with Redis-based distributed locking. The frontend follows clean architecture principles with Riverpod for state management and GoRouter for navigation, ensuring a smooth user experience. Security is enforced through OAuth2 and JWT with PKCE, and a booking-verified review system enhances user trust. Overall, Planymo provides a secure, scalable, and efficient solution for modern travel booking with strong potential for future enhancements.

1.2 SCOPE OF WORK

This project focuses on the design and development of Planymo, a scalable and high-performance travel marketplace platform that integrates mobile clients, vendor dashboards, and a distributed backend infrastructure. The scope includes the implementation of multiple functional modules such as Authentication & Identity Management, Package Discovery, Booking & Reservation, Payment Processing, Vendor Management, Reviews, and Support Services. The frontend is developed using Flutter to support cross-platform mobile applications, while the backend is built using Django REST Framework-based microservices, each responsible for a specific domain. The system is designed to handle real-time interactions, high user concurrency, and seamless data flow between services.

In addition, the project emphasizes system reliability, scalability, and performance optimization through the use of modern technologies such as Apache Kafka for event-driven communication and Redis for distributed locking. Data consistency is ensured using the Transactional Outbox Pattern, while Docker and Kubernetes are used for efficient deployment and scaling. The scope also includes secure authentication using OAuth2 and JWT, along with UI/UX design focused on responsive and user-friendly interfaces. Overall, Planymo aims to provide a robust, secure, and efficient solution for managing end-to-end travel booking operations in a modern digital environment.

1.3 AIM AND OBJECTIVES OF THE PROJECT

The primary aim of this project is to design and develop Planymo, a scalable and secure travel marketplace platform that connects travelers with service providers through a unified digital ecosystem. The application is intended to simplify the travel booking process by integrating discovery, reservation, payment, and review systems into a single platform. It addresses key challenges such as fragmented services, lack of trust, and inefficiencies during high user traffic. The core objective is to deliver a high-performance system capable of handling concurrent bookings while ensuring data consistency, reliability, and a seamless user experience.

To achieve this aim, the project focuses on implementing a microservices-based architecture where each core functionality such as authentication, package discovery, booking, payment, reviews, and support is developed as an independent service. The system utilizes Flutter for cross-platform mobile development and Django REST Framework for backend services. Advanced techniques such as Apache Kafka for event-driven communication, the Transactional Outbox Pattern for data consistency, and Redis-based distributed locking for concurrency control are incorporated. Additionally, the platform ensures security through OAuth2 and JWT with PKCE, and enhances user trust with a booking-verified review system. The modular and scalable design also allows for future enhancements such as AI-based recommendations, multi-language support, and real-time user interaction features.

CHAPTER 2

LITERATURE SURVEY

Recent advancements in travel and e-commerce platforms highlight the growing importance of scalable and reliable digital marketplaces. In [1], a microservices-based travel booking system is introduced, emphasizing modular service design and independent scalability. The study demonstrates how separating core functionalities such as booking, payment, and user management improves fault isolation and system performance. This approach directly supports the architectural foundation of Planymo, where each module operates as an independent service to ensure flexibility and resilience.

An event-driven system using Apache Kafka is explored in [2], where asynchronous communication between distributed services significantly enhances system responsiveness and throughput. The research shows that decoupled communication reduces latency during peak traffic and ensures reliable event propagation. This concept is applied in Planymo to manage booking confirmations, notifications, and review updates without blocking core operations. Additionally, [3] presents the use of the Transactional Outbox Pattern to maintain consistency in distributed systems by synchronizing database updates with event publishing. This study validates Planymo's approach to ensuring data integrity across multiple services. In [4], a secure authentication framework based on OAuth2 and JSON Web Tokens (JWT) is discussed, highlighting the importance of token-based access control and secure session management in modern applications. The inclusion of Proof Key for Code Exchange (PKCE) further strengthens mobile security. This research forms the basis of Planymo's authentication module.

Furthermore, [5] examines the use of Redis for distributed locking in high-concurrency environments, demonstrating its effectiveness in preventing race conditions and resource conflicts. This directly supports Planymo's booking system, where distributed locks ensure accurate availability and prevent double-booking during simultaneous transactions. Overall, these studies collectively provide a strong theoretical and practical foundation for the design and implementation of Planymo. They validate the use of microservices architecture, event-driven communication, secure authentication mechanisms, and concurrency control techniques, all of which contribute to building a scalable, reliable, and efficient travel marketplace platform.

CHAPTER 3

SYSTEM DESIGN

3.1 ARCHITECTURE DIAGRAM

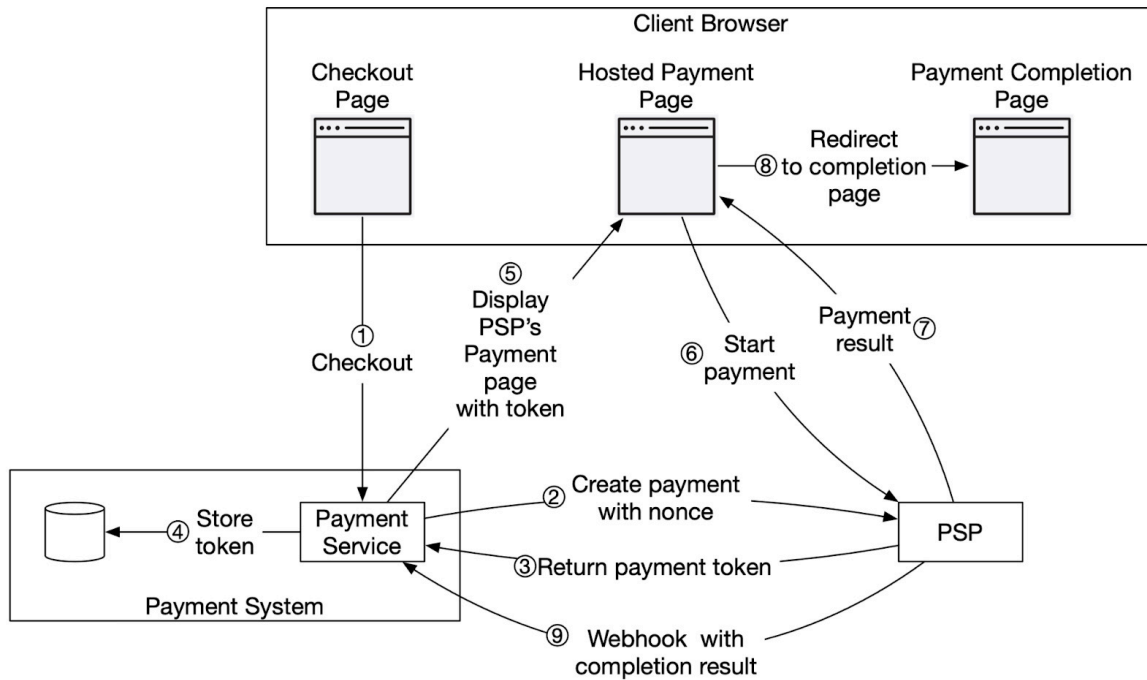


Fig 3.1.1 – Architecture Diagram

The Planymo system follows a layered microservices architecture designed for scalability, performance, and reliability. The presentation layer consists of a Flutter mobile application and a Next.js web dashboard, which handle user interactions and communicate with backend services through REST APIs. All requests pass through an NGINX API Gateway that manages routing, load balancing, and security, ensuring controlled access to the system.

The backend is composed of independent microservices such as authentication, booking, payment, and search, each with its own PostgreSQL database for better isolation and scalability. Apache Kafka enables event-driven communication between services, while the Transactional Outbox Pattern ensures data consistency. Redis is used for caching and distributed locking to prevent issues like double-booking. The system is deployed using Docker and Kubernetes, allowing efficient scaling and high availability.

3.2 USE CASE DIAGRAM

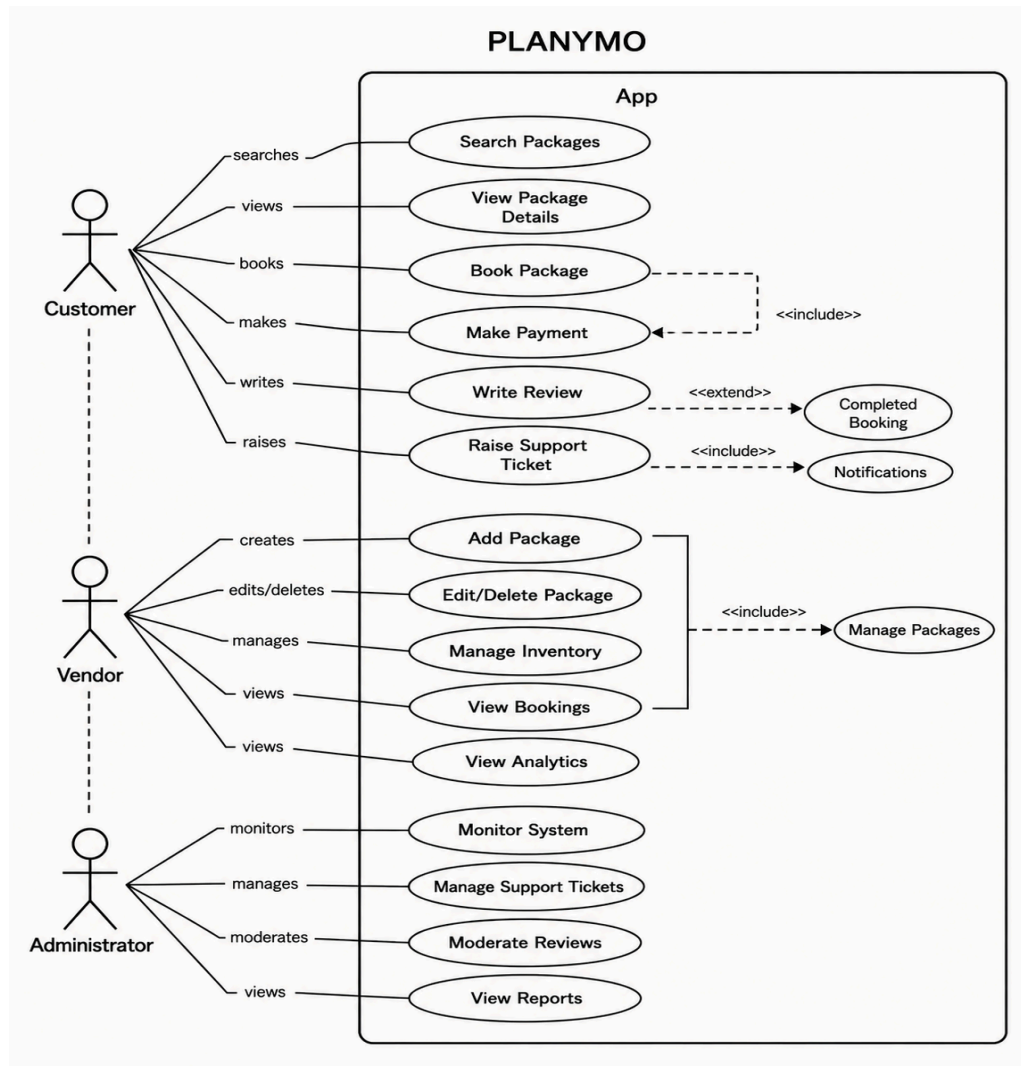


Fig 3.2.1 – Use Case Diagram

The Use Case Diagram for Planymo illustrates the key functionalities of the travel marketplace system involving three main actors: Customer, Vendor, and Administrator. Customers can search and view travel packages, make bookings, complete payments, write reviews after successful trips, and raise support tickets. Vendors are responsible for adding, updating, and managing travel packages, as well as monitoring bookings and viewing analytics. Administrators oversee the system by monitoring performance, managing support tickets, moderating reviews, and generating reports.

These functionalities work together to provide a seamless and efficient travel booking experience. The inclusion of relationships such as booking with payment and review after completion ensures logical flow and system integrity. Overall, the system is designed to be user-friendly, reliable, and capable of handling end-to-end travel operations effectively.

3.3 HARDWARE SPECIFICATIONS

The Planymo system is designed to operate efficiently across standard user devices and cloud-based server infrastructure without requiring specialized hardware. The frontend mobile application runs smoothly on modern smartphones, while the backend microservices are deployed on scalable cloud environments to handle high user traffic and concurrent bookings. The architecture ensures optimal performance, reliability, and scalability across both client and server sides.

Recommended Hardware Specifications:

- **Client Devices (Mobile/Web):**
 - Processor: Octa-core processor (e.g., Snapdragon 720G or higher)
 - RAM: Minimum 4 GB (Recommended: 6 GB for better performance)
 - Storage: 64 GB or higher
 - Display: Touchscreen with standard resolution support
 - Operating System: Android 9.0+ / iOS 13+ / Modern Web Browser
- **Server Infrastructure (Cloud):**
 - Processor: Multi-core cloud instances (e.g., AWS EC2 t3.medium or higher)
 - RAM: Minimum 8 GB (scalable based on load)
 - Storage: SSD-based storage for fast data access
 - Network: High-speed internet connectivity for real-time communication
 - Additional Components: Redis (caching), Apache Kafka (event streaming), PostgreSQL (databases)

These specifications ensure smooth user interaction on the client side and efficient processing, scalability, and high availability on the server side, enabling Planymo to handle real-time bookings and large-scale traffic effectively.

3.4 SOFTWARE SPECIFICATIONS

The Planymo platform is built using a modern, modular software stack to ensure scalability, performance, and seamless user experience across devices. The system integrates multiple components such as travel package discovery, booking, payment processing, vendor management, and support services, all powered by a microservices architecture.

Software Stack:

- **Frontend Framework:** Flutter (for cross-platform mobile development on Android and iOS), Next.js (for web-based vendor dashboard)
- **UI/UX Design:** Material Design 3 for responsive and user-friendly interfaces
- **Backend Framework:** Python with Django REST Framework (for building independent microservices)
- **Database:** PostgreSQL (separate database for each microservice to ensure isolation and scalability)
- **Caching & Concurrency:** Redis (for caching and distributed locking mechanisms)
- **Messaging System:** Apache Kafka (for event-driven communication between services)
- **Authentication & Security:** OAuth2 with JSON Web Tokens (JWT) and PKCE for secure access control
- **DevOps & Deployment:** Docker (containerization) and Kubernetes (orchestration and scaling)
- **Testing Tools:** PyTest (backend testing), Flutter Test (mobile testing), Playwright (end-to-end web testing)

This software stack ensures efficient data handling, secure communication, and high system reliability, enabling Planymo to handle real-time operations and large-scale user traffic effectively.

CHAPTER 4

PROPOSED SYSTEM

The proposed system for Planymo is designed to provide a unified and scalable travel marketplace platform that seamlessly connects travelers with service vendors. It integrates multiple core functionalities such as package discovery, booking and reservation, secure payment processing, vendor management, reviews, and support services into a single cohesive system. The platform focuses on delivering a user-centric experience by ensuring smooth navigation, real-time updates, and reliable transaction handling, even during high traffic conditions.

Key components of the proposed system include:

1. **Package Discovery:** Enables users to search, filter, and explore travel packages based on preferences such as location, price, and ratings.
2. **Booking & Reservation:** Allows users to securely reserve travel packages with real-time availability tracking.
3. **Payment Integration:** Supports secure transactions through multiple payment methods with webhook-based confirmation.
4. **Vendor Management:** Provides vendors with tools to create, update, and manage travel packages and inventory.
5. **Booking-Verified Reviews:** Ensures only users with completed bookings can submit reviews, enhancing trust and authenticity.
6. **Support System:** Allows users to raise and track support tickets with real-time updates.

The system integrates these modules using a microservices architecture, ensuring each component operates independently while maintaining seamless interaction through event-driven communication using Apache Kafka. Data consistency is achieved through the Transactional Outbox Pattern, and Redis is used for caching and distributed locking to prevent issues like double-booking. The frontend is developed using Flutter for cross-platform compatibility, while the backend uses Django REST Framework. Deployment is managed using Docker and Kubernetes, ensuring scalability, reliability, and efficient resource utilization.

CHAPTER 5

MODULE DESCRIPTION

The Planymo system is designed with multiple modules to provide a seamless and efficient travel marketplace experience. Each module is developed as an independent microservice, ensuring scalability, reliability, and smooth interaction across the platform.

1. Authentication & Identity Module

This module manages user registration, login, and secure session handling. It uses OAuth2 with JWT and PKCE to ensure safe authentication. It also supports token refresh mechanisms for continuous user sessions without repeated logins.

2. Package Discovery Module

The Discovery module allows users to search and explore travel packages based on filters such as location, price, and ratings. It provides real-time search results and supports sorting and categorization for better user experience.

3. Booking & Reservation Module

This module handles the core booking process. Users can select and reserve travel packages with real-time availability tracking. Redis-based distributed locking is used to prevent double-booking during concurrent transactions.

4. Payment Integration Module

The Payment module processes secure transactions through multiple payment methods. It follows a webhook-based confirmation approach to ensure bookings are validated only after successful payment completion.

5. Vendor Management Module

This module enables vendors to create, update, and manage travel packages. It also provides features to manage inventory, upload package details, and monitor bookings and performance.

6. Booking-Verified Review Module

The Review module ensures that only users who have completed bookings can submit reviews. This enhances trust and prevents fake or misleading feedback on the platform.

7. Support & Analytics Module

This module allows users to raise support tickets and track their status. It also provides administrators with system monitoring tools and analytics for better decision-making and issue resolution.

5.1 KEY BENEFITS OF THE PROPOSED SYSTEM

Scalable and Reliable Architecture:

Planymo utilizes a microservices-based architecture that allows each component to scale independently. This ensures high performance and system reliability, even during peak user traffic.

Efficient and Secure Booking System:

The platform provides real-time booking with Redis-based distributed locking to prevent double-booking. This guarantees accurate availability and a smooth reservation experience.

Event-Driven Communication:

With Apache Kafka, the system enables asynchronous communication between services, improving responsiveness and reducing system dependency during heavy operations.

Enhanced User Trust:

The booking-verified review system ensures that only genuine users can provide feedback, increasing the credibility and reliability of reviews.

Secure Authentication Mechanism:

OAuth2 with JWT and PKCE ensures strong security, protecting user data and enabling safe access across the platform.

Cross-Platform Accessibility:

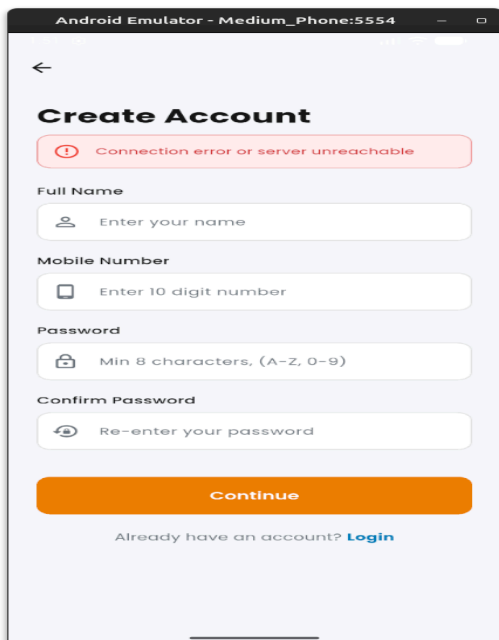
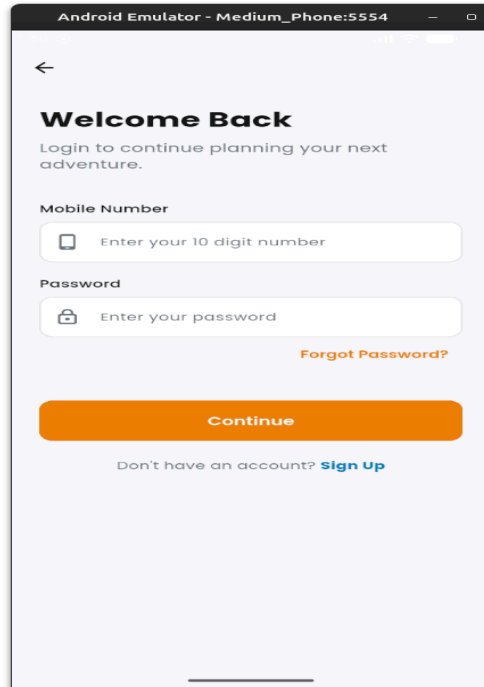
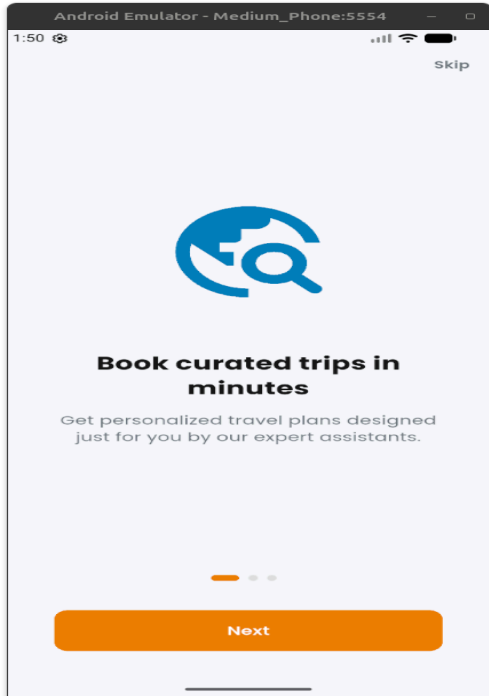
The Flutter-based mobile application and web dashboard provide a consistent and user-friendly experience across devices.

High Performance and Scalability:

Deployment using Docker and Kubernetes allows automatic scaling, load balancing, and high availability, making the system capable of handling large-scale operations efficiently.

CHAPTER-6

OUTPUT



CHAPTER 7

CONCLUSION AND FUTURE ENHANCEMENTS

Planymo provides users with a comprehensive travel marketplace solution that simplifies the process of discovering, booking, and managing travel packages. With features such as real-time package search, secure booking and payment processing, vendor management, booking-verified reviews, and support services, the platform ensures a seamless and reliable user experience. Its microservices-based architecture and event-driven communication enable high performance, scalability, and efficient handling of concurrent users, making it suitable for modern travel demands.

Future Enhancements:

1. **Personalized Recommendations:** Implement AI-based recommendation systems to suggest travel packages based on user preferences and past activity.
2. **Multi-Platform Expansion:** Extend support to additional platforms such as web-based customer portals and broader device compatibility.
3. **Real-Time Communication:** Introduce in-app chat features for direct interaction between travelers and vendors.
4. **Multi-Language and Currency Support:** Enable localization features to support global users with different languages and currencies

CHAPTER 8

REFERENCES

- [1] M. S. Kleppmann, *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. O'Reilly Media, 2017.
- [2] Apache Software Foundation, “Apache Kafka Documentation,” 2024. [Online]. Available: <https://kafka.apache.org>
- [3] Django Software Foundation, “Django REST Framework Documentation,” 2024. [Online]. Available: <https://www.django-rest-framework.org>
- [4] Google, “Flutter Documentation,” 2024. [Online]. Available: <https://docs.flutter.dev>
- [5] Redis Labs, “Redis Documentation,” 2024. [Online]. Available: <https://redis.io>
- [6] OAuth Working Group, “OAuth 2.0 Authorization Framework,” IETF RFC 6749, 2012.
- [7] Kubernetes Authors, “Kubernetes Documentation,” 2024. [Online]. Available: <https://kubernetes.io>
- [8] Docker Inc., “Docker Documentation,” 2024. [Online]. Available: <https://docs.docker.com>